

Building an ML team of agents using ADK

Designing a harness for agents that decide: authority, observation, communication, and consequence

Most "AI agent" projects are pipelines wearing a costume. This is an account of building one, noticing, and rebuilding it properly on Google's Agent Development Kit — and of what ADK gives you that a plain agent library does not.

What is an agent, really?

Strip away the marketing and an agent is three things:

- 1 **A goal** — stated in language, not in code.
- 2 **Tools** — ordinary functions it may call. *Read the spreadsheet. Train a model. Save a file.*
- 3 **The right to choose** — it decides which tool to use next, and when it is finished.

The third one is the whole definition. Everything else is plumbing.

Here is the test. Take your system and delete the instructions that tell the model what to do next. If the output gets worse but keeps going, you have agents. If it stops dead, you have a pipeline with a language model reading the steps aloud.

We failed that test. More on that later, because the failure is more instructive than the success.

Agent vs. pipeline

	Pipeline	Agent
Next step	fixed in advance	chosen at runtime
Failure	crashes or returns nothing	notices, and tries something else
Cost	predictable	bounded only by what you enforce
Demo	flawless	messy

That last row is the trap. **Pipelines demo better, always** — because a pipeline cannot surprise you. Every property that makes a demo go smoothly is a property you get by taking decisions away from the agents. Which is why so many systems quietly become pipelines somewhere between the whiteboard and the repo.

What is Google ADK?

The **Agent Development Kit** is Google's open-source framework for building and running agents. It is easiest to understand in two halves.

The authoring half — the part every agent library has:

- **LlmAgent** — one agent: a model, an instruction, a set of tools.
- **Workflow agents** — Sequential, Parallel, and Loop containers for composing agents into a structure.
- **Tools** — plain Python functions, with their signatures turned into model-readable schemas automatically.
- **Transfer** — one agent handing control to another.

The runtime half — the part that is actually hard, and the reason this article exists:

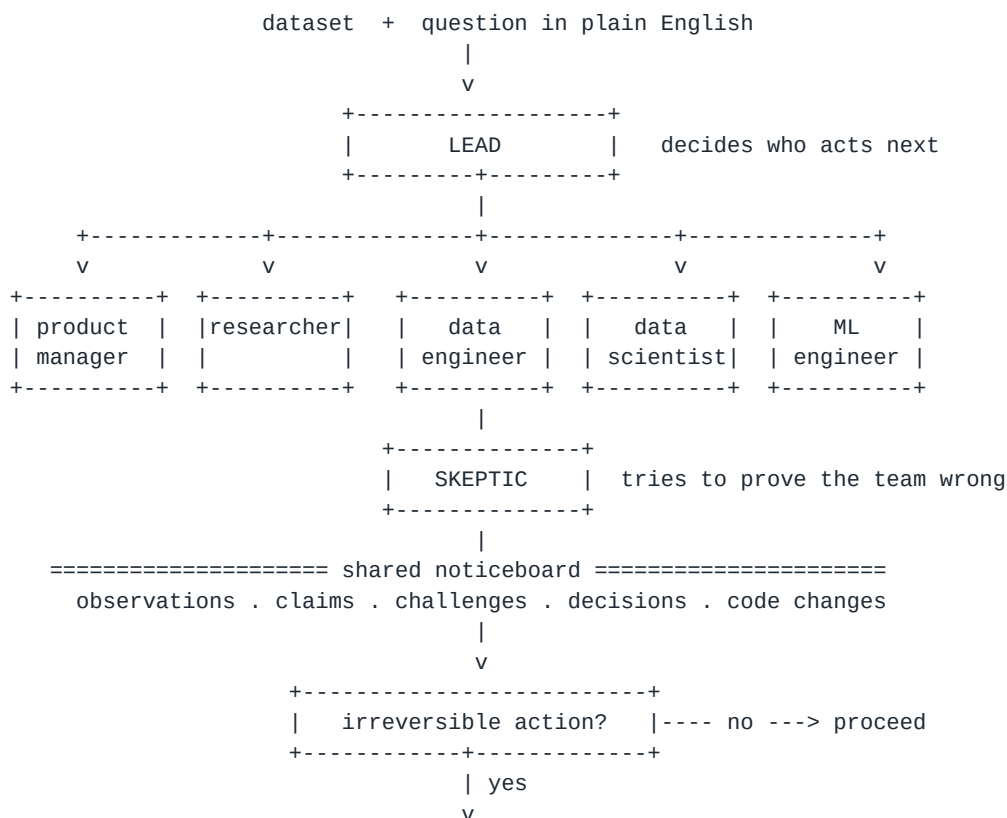
- **Sessions** — conversation and event history, persisted (in-memory, SQLite, or a database).
- **Plugins** — one object that intercepts *every* tool call from *every* agent in the tree.
- **Tool confirmation** — pause a tool mid-flight and wait for a human.
- **Resumability** — an interrupted run continues instead of replaying.
- **Callbacks** — hooks before and after each agent turn and each model call.

Plenty of libraries give you the first half. The second half is the difference between a demo and something you would let near production data. We will come back to why that matters.

What this PoC is about

A team of agents that does machine learning.

You hand it a dataset and a question in plain English — *which customers are about to cancel?* — and a team of specialists works the problem the way a real team would: frame it, study the data, clean it, engineer features, choose and train a model, argue about whether the result is trustworthy, and hand back something a human can sign off on.



ask a human, and WAIT

Two things in that picture are the point of the whole project.

The **noticeboard** — every agent reads it on arrival and posts to it before leaving. It is how a finding reaches someone who never asked for it.

The **wait** — the system asks a human only before something it cannot undo, and then it *holds*, rather than shutting down and resuming later.

The machine learning here is deliberately modest. The thing under construction is the **harness**: the layer that makes agents with real freedom safe to run.

Designing the system

Once you stop scripting agents, you need a different set of guarantees. We landed on four questions, and every design decision below answers one of them.

	The question it answers
Authority	What can this agent decide on its own?
Observation	What can it do to find things out?
Communication	How does it reach a colleague who knows more?
Consequence	What can it never do without a human?

Authority: a charter, not a checklist

The failed version of this project appended a line to every model request:

```
NEEDED: call:clean_dataset
Follow NEEDED exactly.
```

Each agent was handed its next move, every turn. The runs were flawless, because nothing in them was ever in question. That line is deleted, and so is the fixed stage list, and so is a variable called `strategy_level` that let the "improvement loop" choose between exactly four outcomes.

What replaces them is not a better script. It is a job description. Every agent now gets four sections:

- **Charter** — what you are responsible for
- **Authority** — what you decide alone, without asking
- **Escalation** — what you must never decide alone
- **Obligations** — what you owe your colleagues before moving on

That is what you hand a senior engineer on day one, and it is all they need.

Observation: agents that write the code

An agent's power ends at the edge of the tools you thought to give it. Ours had a "feature engineering agent" that knew exactly three tricks — all three hand-written by us, for one specific dataset. It could not invent anything.

So in the new design, the modeling code is demoted from *product* to *starting point*. Every agent can read, write, edit and run real Python in a workspace that belongs to the experiment. The data engineer writes the cleaning script. The ML engineer writes the training script. None of it ships in the repository.

Communication: three channels

Handoff, consultation, and the noticeboard. A team that can only hand off is a relay race — nobody gets a second opinion. A team that can only consult never finishes anything. Details below.

Consequence: gate on irreversible, not expensive

Our first version asked permission before *training a model*. Training is cheap and completely reversible. So the gate interrupted a person without protecting anything — which is the worst outcome, because it teaches reviewers to click approve without reading.

No approval needed	Requires a human
reading, profiling, sampling data	promoting a model to production
writing or editing code	publishing an interface others depend on
training a candidate to see how it does	signing the final decision record
running code in the sandbox	overwriting the source data
posting to the noticeboard	anything that spends money or leaves the building

Deep dive: orchestration

The lead

At the top sits an agent whose only output is a choice of who acts next. It does no analysis, writes no code, and trains nothing. It reads the noticeboard, looks at the goal, and routes.

This is the piece people skip, and it is what separates a swarm from a free-for-all. Someone has to be accountable for sequencing, even when the sequence is not known in advance.

Handoff vs. consultation

ADK gives two distinct mechanisms, and the difference is not cosmetic:

Handoff (`transfer_to_agent`) — "*this is your problem now.*" Responsibility moves. The first agent is finished. Available automatically once agents are siblings under a coordinator with peer transfer enabled.

Consultation (`AgentTool`) — wraps a whole agent as an ordinary tool. "*What do you make of this?*" The colleague answers; control returns immediately. Responsibility does not move.

One structural constraint shapes everything: in ADK, an agent belongs to exactly one parent. So the specialists sit in a **flat pool** under the lead, and all consultation flows through `AgentTool` rather than through nesting.

The noticeboard

Point-to-point messaging is a phone system, not a team. The third channel is an append-only log in the experiment's own storage. Every agent reads it on entry and writes to it on exit:

```
{
  "author":    "data_engineer",
  "kind":      "observation",      # claim | challenge | decision | note | code_change
  "body":      "8,412 rows share a patient id with another row.",
  "evidence":  {"duplicate_ids": 8412, "script": "audit/dupes.py"},
  "refs":      ["hyp-03"],
}
```

Two reasons it earns its cost. It is the only structure that lets an agent be **influenced by work it never requested** — which is most of what makes a team better than five freelancers. And because every entry carries its evidence, "the agents collaborated" becomes a document a reviewer can audit instead of a claim in a slide deck.

The skeptic

Freedom without opposition is not exploration. It is wandering.

So there is an agent whose entire charter is to prove its own team is cheating. In machine learning the most common way to get a spectacular result is **leakage** — the model accidentally sees the answer. A column that encodes the outcome. The same patient in both the training set and the test set. Rows that could never have been positive, left in to flatter the score.

Each of those produces a beautiful number and a worthless model. The skeptic hunts them and returns **verdicts with evidence**, not scores. A confirmed leak sends the work back with a written reason, posted where everyone can see it.

This is also what gives the retry loop something real to do. The old loop swapped a model type. The new loop has to survive an audit.

Hypotheses instead of dials

Every attempt files a record *before* it runs:

```
{"change": "drop hospice discharges", "rationale": "...",
 "prediction": "recall falls ~4 points, and the drop is honest",
 "outcome": None, "verdict": None}
```

Stating the expectation up front and scoring it afterwards is the difference between a loop that repeats and a loop that **learns inside a single run**. It is also the cheapest readable summary for the human following along.

Deep dive: tools

A tool is just a function

In ADK, you write an ordinary Python function with type hints and a docstring, and the framework turns it into something the model can call. The docstring is not decoration — it is the specification the model reads to decide whether this tool is the right one.

The workspace

Instead of a fixed catalogue of ML operations, every agent gets the same small, general set:

```

read_module(path)
list_workspace()
write_module(path, source, rationale)    # commits; announces to peers
patch_module(path, old, new, rationale)  # any agent, any file, any time
run_module(path, argv) -> {stdout, stderr, exit_code, artifacts, duration}

```

Everything the agents write lands in **version control**. Which means *"the AI changed the code"* stops being a log line you have to trust and becomes a **diff** — with a justification the agent was required to write, reviewed the way you review any colleague's pull request.

We spent a long time designing audit trails for this system. The best one turned out to have been invented in 2005.

What "sandboxed" has to mean

Letting a model write and execute code deserves more than a reassuring adjective. Three layers, and it is worth being honest about which are real:

- 1 **Inspect before running.** Parse the code, allow the data-science libraries, and reject anything touching the network, the operating system, or files outside the workspace.
- 2 **Isolate the execution.** A separate process, working directory pinned to the workspace, hard wall-clock and memory limits, so a runaway script dies on its own.
- 3 **Cut off the network.** Genuinely requires a container. Our proof of concept does not have it yet.

Two of those are enforcement. One is a plan. Saying so is more useful than saying "sandboxed."

Tool confirmation

ADK lets you mark a tool as requiring human confirmation — and, importantly, lets that be a **function** rather than a flag. So "does this need a human?" is evaluated at call time, against the actual arguments and the current state. *Promoting this model to production needs a human; training a fourth candidate does not.*

How ADK helps at runtime

This is the half that is hard to build yourself, and the reason the project sits on ADK at all.

```

+-----+
|          YOUR AGENTS          |
+-----+
|  PLUGIN  - every tool call, every agent | <- the harness
|  limits . rules . records . gates      |
+-----+
|  ADK RUNTIME                    |
|  sessions . confirmation . resumability |
+-----+
|  MODEL  (Gemini, or anything via LiteLLM) |
+-----+

```

Sessions that outlive the process

An agent run is a long conversation. ADK persists that history — every message, tool call and result — so a crash, a deploy, or a human going to lunch does not erase the run. Swap the backend from in-memory to SQLite to a managed database without touching agent code.

One plugin sees everything

This is the feature the harness is built on. A plugin is a single object installed on the app that intercepts every tool call from every agent in the tree — before and after. You do not wrap tools, decorate functions, or trust each agent to report honestly.

In roughly a hundred lines, that one hook gives you:

- **Records** — every call, its arguments, who made it, how long it took, whether it failed.
- **Rules** — return an error object instead of running, and the tool is simply skipped.
- **Checkpoints** — snapshot the run's state before anything happens.
- **Limits** — count calls, fits, wall-clock, bytes, and stop at the ceiling.

The plugin does not know what a feature or an F1 score is. It governs whatever the agents happen to be doing, which is exactly the property you want when the agents can rewrite their own code.

Pausing for a human, properly

This is where we had it most clearly wrong, and where ADK gives you the pieces to fix it.

The naive version: pause, save state, exit the process. The human approves an hour later, the system restarts and rebuilds from disk.

Think about what that costs. The answer arrives in a different world from the one that asked the question. Everything the agent was holding in its head but had not written down is gone.

The version we want: the agent asks, and the system *holds* — session alive, reasoning intact, nothing unwound:

```
decision = await broker.request(
    kind="promote_model",
    question="Promote iter3 (recall 0.71 at the cost-optimal threshold)?",
    options=["approve", "reject", "redirect"],
    evidence={...},
)
# Execution parks here. The agent's turn never unwinds.
```

And note the third option. **A decision is not a boolean.** `redirect` carries a note — *"you are testing on the same patients you trained on, go fix that"* — which returns as the tool's result *and* lands on the noticeboard for every colleague to see. The agent absorbs it and re-plans on the spot.

That is the difference between a human in the loop as an **approval queue** and a human in the loop as a **colleague**.

ADK's built-in confirmation and resumability do not disappear — they get demoted to what they are genuinely excellent at: durability for when the process really does die.

Swappable models

The same agent tree runs on Gemini, on anything reachable through LiteLLM, or on a scripted in-process model that follows instructions exactly. That last one matters more than it sounds: **the same graph, with a deterministic model, gives you a repeatable integration test** for a system that is otherwise stochastic by design.

Why ADK is more than an agentic library

Most agent libraries are competing on the authoring half — a nicer way to define an agent, compose a chain, register a tool. That half is close to solved, and it is not where projects fail.

Projects fail on the questions that only appear once agents have real freedom:

- What happened, exactly, and in what order, three weeks ago?
- Who approved this, on what evidence, and how long did they take?
- The process died mid-run. Do we lose the work?
- This agent is about to do something irreversible. What stops it?
- We have spent \$400. What is the ceiling?

You can build answers to all of that yourself. Teams do, badly, twice, because the first version lives inside the agent definitions and the second one discovers that governance belongs *underneath* them, not alongside.

ADK's contribution is that the intercept point, the session store, the confirmation protocol and the resume path already exist and sit **below** your agents. That is what makes the harness a hundred lines instead of a subsystem — and it is why "agent library" undersells it. **The library part is how you write agents. The runtime part is how you are allowed to run them.**

And here is the reversal worth ending on. Build all of that for a system whose next step is already decided, and reviewers will reasonably ask why you bothered. Point it at agents that can rewrite their own training code, and the same machinery is the bare minimum.

The harness was never the supporting cast. We just had not yet built anything that needed it.

Where this stands

Honest accounting, because the two halves of this article are at different stages.

Working today: the agent tree, the plugin with records and rules and checkpoints, persisted sessions, tool confirmation, resumability, a web service and interface, and the deterministic test model.

Being built, in this order: the hold-in-place decision broker; the version-controlled workspace and the execution boundary; the noticeboard and the swarm wiring; the charters and the skeptic. The deletions — the **NEEDED:** line, the fixed stages, the `strategy_level` dial — come last, once nothing depends on them.

The new version is going up **beside** the old one rather than on top of it, so there is always something that runs and the two can be demonstrated side by side.

That comparison is the argument. One of them will give a beautiful demo every single time.

We are betting the other one is worth more.